

Strengths and barriers behind the successful agile deployment—insights from the three software intensive companies in Finland

Minna Pikkarainen · Outi Salo · Raija Kuusela ·
Pekka Abrahamsson

Published online: 27 October 2011
© Springer Science+Business Media, LLC 2011
Editor: Laurie Williams

Abstract The number of success stories being reported concerning agile software development has led to an increase in interest among industries and research communities. The purpose of this paper is to identify strengths and barriers for ‘successful agile deployment’ in the software companies. This knowledge can benefit software companies planning their current strategy for agile deployment. Analysis of 57 developers, architects, project managers, customers, quality managers, and line and product managers in three case companies identifies 71 strengths and 169 barriers of agile deployment. The analysis revealed the importance of management providing the necessary goals and support for agile development. It also indicated the significance of defining a tailored process model and giving developers the freedom to improve their own agile development process continuously during agile deployment. The identified barriers, strengths and recommendations can be used as a checklist for planning and/or monitoring the effectiveness of agile deployment in software companies. By identifying the barriers and strengths of agile deployment, the paper deepens understanding of this highly relevant but relatively under-researched phenomenon and contributes to the literature on agile deployment and software process improvement.

Keywords Agile deployment · Improving agile development · Success factors for agile deployment · Software process improvement (SPI)

M. Pikkarainen (✉) · R. Kuusela
VTT Technical Research Centre of Finland, P.O. Box 1100, FIN-90571 Oulu, Finland
e-mail: minna.pikkarainen@vtt.fi

R. Kuusela
e-mail: raija.kuusela@vtt.fi

O. Salo
Nokia Corporation, Oulu, Finland
e-mail: outi.salo@nokia.com

P. Abrahamsson
The University of Helsinki, Helsinki, Finland
e-mail: pekka.abrahamsson@cs.helsinki.fi

1 Introduction

Agile methods in software development have been increasingly adopted since the 1990s. The philosophy of agile software development is based on the values and principles identified in the Agile Manifesto (Hansson et al. 2006). Agile software development includes methods such as Extreme Programming (XP) (Beck and Andres 2005) and Scrum (Schwaber and Beedle 2002), which consist of a group of agile practices. It has been claimed that *agile development* methods would increase the ability of software organisations to respond to dynamic market changes (Kettunen 2009). Since the mid 1990s, agile methods have allowed software-intensive companies to achieve their targets in decreasing lead times and improving the quality of their products. However, deployment of agile methods demands the acquiring, assimilating, transforming and exploiting of new knowledge (Cohen et al. 2004). It is no surprise, therefore, that several studies report the deployment of agile software development methods being a challenging activity (Svensson and Höst 2005; Chan and Thong 2008). According to Boehm and Turner (2003), *plan-driven software development* is an engineering approach in which the software is developed following specific processes, starting from requirements and ending with the finished code. They claim that there are several methodological approaches and models for how to develop the software in a plan-driven way. The best-known of these approaches is probably the ‘waterfall’ model (Royce 1970), in which all the development phases (e.g. requirements definition, implementation and testing) are implemented at least twice during the long time period, one after the other, in order to produce a working software solution. Recent studies have investigated the pros and cons of plan-driven and agile processes. Petersen and Wohlin (2010) have researched the effect of moving from a plan-driven development process to an agile one. They claim that agile practices bring added value compared to the plan-driven approach. Issues requiring further improvement were also found, however.

Agile deployment is the process that occurs within the organisation when new agile development methods are taken into use in product development. The variables of agile deployment can be reflected from the innovation diffusion theory described by Fishman (1992). According to Fishman (1992) the innovation is still in deployment phase as long as:

- It is still new for the adopters
- It is not yet part of the organizational routines and process model
- It is not yet achieved a critical mass with a community of users

Agile deployment starts when the agile methods or practices are first time (when they are still new for their adopters) used in selected pilot projects and are still new to the development team, managers and quality engineers of the organization. The deployment ends 1) when the agile methods are used as a regular, daily activity in each project and as a basic activity that is integrated as a part of the organizational process model and 2) when the agile methods are used by the large amount of projects in the company.

Various approaches can be taken when deploying new methods. Plan-driven software development permits revolutionary (Sweeney and Bustard 1997) and evolutionary (McFeeley 1996) approaches. In addition, different models are traditionally proposed to support the continuous improvement of organisational software development processes, e. g., IDEALSM (McFeeley 1996) and Quality Improvement Paradigm (QIP) (Basili 1989). Agile principles address continuous improvements; short iterations provide an opportunity for the rapid improvement, fast feedback, piloting and validation of new methods in projects and within an organisation (Salo and Abrahamsson 2007). Subsequently, it is

proposed that QIP could be modified to include an agile-specific perspective in development (Pikkarainen et al. 2005).

In Software Process Improvement (SPI) literature, the success in deployment of improvement suggestions is typically analysed through several impact factors (Stelzer and Mellis 1998; Dybå et al. 2004; Börjesson et al. 2006). Furthermore, the success of deployment in the context of process improvement may occur when the commitment to the change and implementation effort exist on both organisation and project levels (Dybå et al. 2004).

The importance of the research on the success factors has been noticed also in agile studies in which the agile methods are already in use (deployed) in an organization. For example, the study of Misra et al. (2009) uses a survey of 241 questions to analyse the success factors of agile development (agile practices in use). Based on the analysis, they reveal eight factors that demonstrate a correlation between agile development and success, i.e. customer satisfaction, customer collaboration, customer commitment, decision time, corporate culture, personal characteristics, societal culture, and training and coaching.

In this study few have looked at the very notion of a “Successful Agile Deployment” which in our paper means a success in deployment of agile practices in software company. The objective of this research is to analyse the strengths and barriers in the selected pilot projects in three software engineering organisations, in which the goal is to deploy agile development methods. A further objective is to conclude this analysis with recommendations to support software companies planning agile deployment. In order to achieve these goals, we conducted three in-depth longitudinal case studies in which agile method deployment was investigated in each of three organisations. The case studies and continuous data analysis were conducted over a period of 2.5 years.

Section 2 of this paper presents the background for this study, including a discussion of different agile deployment approaches and steps. Section 3 describes the research approach and context. Section 4 presents the analysis results of the case companies. Section 5 discusses the research results and recommendations. The concluding section contains the limitations of the study and an outline of avenues for future research.

2 Background

This section describes the deployment approaches for both plan-driven and agile software development.

2.1 Evolutionary Vs. Revolutionary Deployment Approaches

The deployment of new methods is a complex task, which requires a great deal of effort. In software engineering, deployment approaches can be categorised as evolutionary or revolutionary. Evolutionary deployment means that improvements are introduced in organisations in small pieces and tested in pilot projects before use on a large scale. Evolutionary approaches are often used with relatively minor or incremental changes, when adapting simple, existing practices. In the revolutionary approach, i.e., the “Big Bang” approach, improvements are effected on a large scale, often requiring major changes in organisational structures and processes. The revolutionary approach has been shown to be a useful mechanism for companies aiming to awaken developers and increase their motivation. (McFeeley 1996; Sweeney and Bustard 1997; Dybå 2005; Stelzer and Mellis 1998; Cohen et al. 2004).

2.2 Process and Steps for Deployment

Software-intensive organisations and research communities have reported on their experiences of agile methods, such as Scrum and XP deployment (Rasmusson 2003; Cohen et al. 2004; Drobka et al. 2004; Kähkönen and Abrahamsson 2004; Fitzgerald et al. 2006; Dybå and Dingsøyr 2008). Piloting seems to be the most popular method of agile deployment in software organisations.

In SPI literature, deployment paradigms such as QIP (Basili 1989), and IDEAL (McFeeley 1996) have been argued as purposefully modifying the behaviour of an organisation (Garvin 2000). The agile deployment process has only rarely been integrated into plan-driven deployment models (Kähkönen 2005; Pikkarainen et al. 2005; Salo and Abrahamsson 2007; McCaffery et al. 2008). Agile principles (Beck et al. 2001) propose that agile project teams should iteratively reflect on how to become more effective. Thus, in agile development, the project-level learning cycle focuses in particular on building and adapting processes for project-specific needs. However, the improvement of organisational base processes should not be forgotten either; integrating rapid, iterative project level reflections into the organisational improvement cycle provides an efficient mechanism for continuously improving the software development process at both project and organisational levels (Salo and Abrahamsson 2007).

QIP consists of a continuous and dual learning cycle of both the corporate and project levels. The continuous corporate learning cycle of QIP includes six phases: 1) characterise and understand, 2) set objectives, 3) choose processes, methods and techniques, 4) execute (project learning cycle), 5) analyse and 6) package. In QIP project learning mainly serves the purpose of organizational improvement and neglects to consider the rapid and continuous learning of individual project teams as claimed in agile principles (Salo 2006).

Although it seems that QIP would provide the required link between the project and organisational level activities, the application of QIP has only been reported in a few cases of agile deployment (e.g. in Salo and Abrahamsson 2007). Table 1 describes the agile deployment steps and the main activities that can be employed by supporting the various process phases of agile deployment. This framework proposes that the deployment steps are either implemented at an organisational or at a project level. For example, agile practice selection, deployment planning and analyse, improve and package are organisational-level activities that require organisational-level decision-making, whereas deployment execution

Table 1 Agile deployment steps and main activities (Pikkarainen et al. 2005).

Agile deployment steps	Main activities
1. Select agile practices	Set goals for the deployment Identify suitable agile practices Select and prioritise the agile practices to deploy
2. Plan the deployment	Plan the deployment Prepare the deployment
3. Execute the deployment	Execute the deployment Iteratively improve, validate and package the feedback in projects
4. Analyse, improve and package	Analyse project practices, problems and recommendations, based on project feedback Improve the organisational processes Package

and iterative improvement, validation and feedback collection require the participation of the project team.

The Selection of Agile Practices In this article, piloting is considered to be first applying agile methods in certain projects, and then adapting the methods to other projects based on what has been learned. In selecting agile practices the main activities are to set objectives, identify suitable agile practices for the pilot projects and then select and prioritise the suitable agile practices for deployment, either in pilot projects or at an organisational level (Pikkarainen et al. 2005). Although the methods for selecting agile practices, or defining the level of agility for a team, or the purposes of a company, are rare, some frameworks or guidelines exist for addressing the agility evaluation (Boehm and Turner 2003; Kettunen and Laanti 2005; McCaffery et al. 2006). The aim in these models is either to evaluate the needs for the introduction of agile methods, or to identify the risks that could emerge from the overall increase in agility.

Assessments are used to obtain the prevailing status of the processes while agile deployment is under way. They help us understand what could be improved and why. Agile assessment (Pikkarainen and Mäntyniemi 2006) and AHAA assessment (McCaffery et al. 2009) are methods that can be used to identify improvement suggestions for the selected software processes, via plan-driven and agile practices.

Plan Deployment Deployment planning is for analysing and prioritising the identified process improvement opportunities in an organisation. The first task is often to select the improvements that are to be piloted or adopted. The second task is to select and tailor the agile process and to set up the required tool environments, for example, for automated testing and continuous integration. Tailoring has been considered a necessity in many companies before any agile methods can actually be executed (Lindvall et al. 2004; Fitzgerald et al. 2006).

Execute Deployment In the execution phase, the agile methods are adopted as planned. The deployment of agile principles may require additions to the daily procedures, such as metrics data collection and improvement workshops. In most cases, the integration of a specific agile method (e.g., XP, Scrum or integration of XP and Scrum) has been adopted in pilot projects (Rasmusson 2003; Cohen et al. 2004; Drobka et al. 2004; Kähkönen and Abrahamsson 2004; Fitzgerald et al. 2006).

Salo and Abrahamsson (2007) write that when adopting agile methods the organisational process improvements can originate from the daily problems and improvement ideas of the development teams. Iterative process adaptation is therefore an important activity for any software organisation. The systematic validation and packaging of project-level improvement actions thus also serves as feedback for the organisational-level improvement.

Analyse, Improve and Package SPI is a continuous process in which the potential improvement opportunities are identified, prioritised and selected in a cyclical manner. The high-priority software process improvements should thus be selected for embedding in the organisational software development practices for the following improvement cycle. This is performed in the analysis, improve and package phase, where the goal is to analyse the deployment results based on the collected feedback, and then to package them as an organisational-level process model. The model, in turn, is continuously updated and improved, based on the new feedback received from the pilot projects.

Current literature either presents approaches that focus on supporting the mapping between agile and plan-driven software development, or introduces a framework or

guidelines for the agility evaluation of the projects (Paulk 2001; Boehm 2002; Kähkönen and Abrahamsson 2004; Pikkarainen and Mäntyniemi 2006). The packaging of agile project experiences is only rarely reported as a systematic company activity. Salo (2006) describes mechanisms for handling information sharing, analysis and packaging between the project and organisational levels. This promotes the continuous improvement of agile software development, using the iterative improvement workshop (i.e. post-iteration workshop) technique and organisational-level workshops.

3 Research Design

3.1 Research Method

This study can be considered as a multiple, in-depth case study: this is because it includes the research data from three case companies, allowing a cross-case analysis and, thus, more general research results when compared to a single-case study (Benbasat et al. 1987). The case-study method (Yin 2003) is a suitable research approach for an overall study in which the researchers act as investigators rather than participants (Benbasat et al. 1987), and where the objective is to investigate a contemporary phenomenon in a real-life context. The case-study approach is also beneficial in researches where control over behaviour is absent owing to research data being collected through observation in an unmodified setting.

3.2 Data Collection and Analysis

Various data collection methods were applied as proposed in the case-study method (Yin 2003). These were dependent on the case-specific deployment process of each company and included such methods as observation, semi-structured group and individual interviews (Robson 2002), and workshops (i.e. traditional project post-mortems), iterative improvement workshops (Salo and Abrahamsson 2007) and agile assessment workshops (Pikkarainen and Mäntyniemi 2006). Altogether, the study included 40 semi-structured interviews (group and individual), 5 post-mortem meetings and 12 improvement workshops (i.e. organisational-level workshops in which the intention was to define the improvement suggestions for agile deployment—see Table 2) conducted during 2005–2008. The data collection involved 57 developers, architects, project managers, customers, quality managers, and line and product managers from the case companies. The data for this study were collected during several research cycles. All data from the 60–90-minute interviews were tape-recorded and transcribed.

Table 2 Data collection

	Case I	Case II	Case III
Semi structured interviews	20	12	8
Post-mortems	1	2	2
Iterative improvement workshops	—	—	5
Improvement workshops	3	4	5
Research period	2006, 2007	2005, 2006, 2008	2005, 2006, 2008
Number of interviewed persons	25	16	16

The role of the researcher in all cases was to assist the companies involved in the adoption of agile methods. All the barriers and strengths described in Tables 5, 6, 7, and 8 were systematically collected from the developers, managers, quality engineers and architects of the three organizations in the 1–3 h post iteration and post project workshops. Firstly, each strength and barrier was written on a piece of paper by the employee of the organization. Secondly, the strength or a barrier was validated in a workshop, where company employees (the project team and managers) and two to three researchers participated. After the validation, a barrier or strength was recorded in the work sheet and categorised together with the whole group. This provided large amount of categorised success factors and barriers that were again validated by the research group using the data collected through the interviews. The case companies studied in this paper are named Case I, Case II and Case III.

In Case I, the research started at the end of the first pilot project in which the goal had been to adopt agile development methods. The target for the research group was to analyse the existing plan-driven approach of the organisation, as well as to evaluate the results of the pilot project post-mortem. To accomplish this, a total of 20 semi-structured interviews were held. The six interviewees from the pilot project consisted of four developers, one tester, one management group representative and one project coordinator, the remaining interviewees being other stakeholders in the organisation. After the analysis, three workshops were conducted to draft a new organisation-specific agile process model in collaboration with the stakeholders. In addition, a project post-mortem session was held for the pilot project team to gather feedback and knowledge from the stakeholders and members of the pilot project.

In Case II, an agile assessment was conducted to select the agile practices for piloting. The assessment included nine semi-structured interviews, and workshops in which ten people from the company participated. These included developers, managers, quality management and the architects of the pilot projects. The deployment-planning phase was supported by conducting one workshop to assist the company in defining a new agile development model, to be piloted in three projects. A second agile assessment was organised after the pilots, including five semi-structured interviews and one workshop. The objective of the second assessment was to analyse the current status after the first pilots and thereby help the company to package their results. In addition, two group interviews were organised with the purpose of defining the barriers for agile deployment within a 2.5 year time period.

In Case III, the research team facilitated five workshops during the organisation's first agile pilot project. Two workshops were held after the pilot, together with the company's developers, management, quality engineers and architects, in order to analyse the needs for the future agile software development process at an organisational level. In addition, one interview was held to gather in-depth information. The research team also held three workshops for the second and third pilot projects in the company, as well as for one project applying a plan-driven process model, in order to define the current state of agile deployment and to share knowledge of agile deployment among the different teams and stakeholders. After a pilot period of 1.5 years, five individual semi-structured interviews and a workshop were organised for the one agile pilot team. Furthermore, the research team participated as observers in the project post-mortem meeting, which involved the participation of all the stakeholders from the project team, including the managers, quality engineers and architects. Finally, in 2008, after 3 years of piloting, two group interviews were held to review all the projects in the company and determine how the situation had changed over time.

The data analysis was performed in continuous collaboration by two researchers, as proposed by Benbasat et al. (1987). The collaboration was intended to provide greater

richness, accuracy and validity in the case results. The results were produced using inductive and deductive analysis (Patton 2002), adopting within-case and cross-case analysis approaches as described by Eisenhardt (1989). The analysis process in this study consisted of two phases, namely 1) within-case analysis (Eisenhardt 1989), in which the strengths and barriers of agile deployment were identified, and 2) cross-case analysis (Eisenhardt 1989), in which the results were compared among the different cases. In the first phase, each of the three cases was analysed in a deductive manner using elements from a QIP-based agile deployment framework. In the second phase, the findings from each case were produced using inductive analysis (Patton 2002), discovering common patterns and themes. The data from the three cases were further analysed in this phase, focusing on the reasons behind the problems that occurred in the case companies, and describing the case similarities and differences. The common patterns and themes which emerged from each case were compared with the success factors and recommendations produced during the second analysis phase.

3.3 Research Context

As the case context is considered critical in case studies, it is important to select the cases and researched entities carefully, in order to make the context easier and limit the analysis process (Miles and Huberman 1999). This can be done, for example, by using specific context factors (i.e. size, domain) when selecting companies for the case-study research (Eisenhardt 1989). As the common denominator, all three case organisations selected for this study are medium-sized enterprises building their own software product. At the beginning of this study all were also about to start adopting agile methods. All the deployment approaches are based on organisational business objectives and decisions, and can be considered as top-down. The case studies were selected for their different implementation of deployment processes and methods, which, again, provides a sound basis for a comparison of the success factors of agile deployment. The case backgrounds are described in Table 3.

This study makes use of an extreme programming evaluation framework (Williams et al. 2004) to introduce the context factors (Table 4). The description of the agile practices used is based on the Fitzgerald et al. (2006) description of the combined, customised use of Scrum and XP practices: Scrum sprints, sprint planning, daily meeting, post-game sessions, a 40-hour week, an onsite customer, pair programming, test-driven development (TDD), continuous integration, collective ownership, simple design, refactoring, coding standards and metaphor.

Case I develops finance-critical products in its internal software development department. The total size of the company is around 8,000 employees. The distribution level of software development activities was low, mainly occurring at one development site. At the time of the study, the organisation applied a well-documented, plan-driven (Boehm and Turner 2003) mode of development. In 2006, the management of the organisation made a decision to deploy agile development methods. The pilot contained two subsequent pilot

Table 3 Case backgrounds

Company	Domain	Size of the company
Case I	Financial	~8,000
Case II	Telecom	~100
Case III	Information security	~300

Table 4 Case context factors

Context factor	Case I	Case II	Case III
Team size	5–7	3–5	5
Team educational level	High	High	High
Domain expertise	Low	High	High
Language expertise	High	High	High
Project manager's experience	Low	High	High
Specialist available	Project manager	None	Architect
Personnel turnover (an example from one case)		(added (5)+(2))/total in the end (7)=100%	(added (1)+left (2))/total in the end (5)=60%
Morale factors	High management push for the pilot project. People working in multiple projects	People were changing between the projects, holidays	Holidays, the competitor already had the system, busy schedule
Agile practices used	Daily Scrums, Pair-Programming	Scrum sprints, sprint planning, daily meeting, post-game sessions, pair programming, continuous integration, collective ownership, refactoring, coding standards	Scrum sprints, sprint planning, daily meeting, post-game sessions, pair programming, continuous integration, collective ownership, simple design, refactoring, coding standards, a 40-hour week, TDD
Effort, in terms of person hours, weeks, or months	~70 person months in total (including SW team, excluding other stakeholders)	First pilot project: 4–5 people/24 months Second pilot project: 4–6 person months	First pilot project: 9 person months Second pilot project: 5–6 people/18 months
Duration, in terms of weeks or months	First pilot project ~7 months, second pilot project ~7 months	First pilot project: 24 months Second and third projects: 2–3 months	First pilot project: 2 months Second pilot project 29 months
Short description of the developed software	Finance critical software for public use	First pilot project: video codec product for mobile applications Second and third pilot projects: customer tailoring for the video codec products	First pilot: mobile version of the internet security system Second pilot: Monitoring system for the internet security
Whether the project/pilot was considered a successful deployment of agile methods or not	No. The results of the pilot post-mortem revealed that the team had failed to achieve the expected degree of success in agile deployment	Yes the deployment was considered successful according to the project managers and site manager	Yes, the managers and developers argued that the deployment was successful
Whether the project/pilot was considered a success from the point of views: 1. Timely delivery, 2. Budget, 3. Quality, 4. Meeting requirements	NA. Clear targets had not been set for the pilot and quantitative data from previous projects was not available	“Productivity: 20% faster than industry average; Increased satisfaction of the project and product managers (improved, more efficient ways to manage the project and its requirements)” manager	“3× reduction in lead-time, 50× better quality, and 5× cheaper” manager

projects mostly with same project members. The first project was launched in autumn 2006 and continued until spring 2007. The second project was launched in autumn 2007 and continued until early 2008. An external manager for the pilot project was hired from a consultant organisation, along with a number of software engineers to support agile deployment. The project contained three external project members and four internal software developers who could all be considered to have a high educational and language expertise level.

The aim of the pilot project was to deploy Scrum. However, a project post-mortem revealed that no agile methods beyond Daily Scrums and Pair-Programming had been applied as such. No evidence concerning systematic iterations or iterative planning could be detected, for example. Practices such as Continuous Integration could not be deployed owing to a lack of appropriate tools. In fact, the team itself identified a “Big Bang” approach, accompanied by a lack of knowledge of agile methods, as among the major causes of failure of the pilot. Despite this, the team still found positive outcomes, including an increased and prompter collaboration of testers, developers and designers, and the greater participation of business stakeholders. The latter was considered to have affected an increased understanding on the part of the developers, as well as more clearly prioritised business needs. Regarding expertise, the team itself evaluated serious deficiencies in its own substance and business domain knowledge. Even with a project manager hired as external expert, the results of the pilot indicated that the team had failed to achieve the expected degree of success. Hence the experience level of the project manager has been evaluated as low.

Case II delivers highly embedded applications and services for telecommunication devices. The company was founded in 1992, and since then has become a market leader in its own specific product development field. In 2008, the case company had 18 semiconductor manufacturer customers in six countries in Europe, Asia and the USA, with development located in Oulu, Finland and New York, USA. At the time research started, the company had around 70 employees, and by the end the number was around 100. Prior to the study, the organisation followed an ad hoc software development process. The deployment of Scrum and XP practices was started in spring 2005, when the company’s senior management heard about the success of agile development in some projects and made a decision to try agile practices in three pilot cases. The first agile pilot project making domain engineering product started during autumn 2005. The project lasted 24 months and the used 2–3 person months work time/each month, on average. The project employees produced a domain engineering; video codec product for the mobile application. During the research (2005–2009), the company tailored XP and Scrum practices also to the others projects. Two of them were small application engineering projects in which the goal was to tailor the previously developed baseline products based on the specific customer needs. These projects were short in terms of the duration (2–3 months on average). The agile practices used by the Case II teams were Scrum sprints, sprint planning meetings, daily meetings, post-game sessions, pair programming, continuous integration, collective code ownership, refactoring and coding standards. Owing to the size of the company the team used was relatively small (3–5) people, although educational level and domain and language expertise were high.

All the project members had worked in the organisation for a period of 3–15 years. Turnover during the projects was high because of the need to move people from platform development to customer-specific projects. In one example project the turnover during the period 2005–2009 was 100%. The transfer of personnel can also be seen as a morale factor that considerably affected the work of the people in the agile teams. According to the

interviewees, all three studied projects were considered successful in terms of the agile deployment. Interviewees mentioned, for instance, that the productivity was 20% faster than industry average due to the agile deployment. Furthermore, satisfaction of the project and product managers improved in the projects compared to the previous situation due to the more efficient ways to manage the project and its requirements.

Case III produces products in the information security field. The corporation produces services for the global market in more than 90 countries. Located in Oulu and Helsinki in Finland, the company also has a large number of regional offices around the world. The number of employees increased continuously during the period of research. At the time research started, the company had around 100 developers, whereas this number grew to around 300 in 2008. In 2005, the case company was developing software using plan-driven software development methods. At this time, the company reported the achievement of level three of the capability maturity model, CMMI and that the quality of the software produced was high in terms of reliability. The first pilot of the combined Scrum and XP practices was launched in 2005. The project lasted 4 months and used 4,5 person months each month on average. The goal of the project was to produce information security application for the mobile devices. In 2008, the company management made a decision to adopt Scrum and XP practices (i.e. Scrum sprints, sprint planning, daily meeting, post-game sessions, pair programming, continuous integration, collective ownership, simple design, refactoring, coding standards, 40-hour week, TDD) in all projects of the company. Piloting was continued directly afterwards with Scrum and some XP practices (i.e. continuous integration, collective code ownership) in two pilot projects. One of them was studied in more detail. It lasted 18 months. The size of the team used averaged 5–6 people. Usually, there were four developers, one quality engineer and a project manager in each team. The team educational level and domain and language expertise were high. Each team member had been working in the company for a period of 2–8 years. Turnover of the personnel varied among the different company sites, at one site 60%, for example, and 20% at another. One team also had a specialised architect to support them during the entire development. Holidays and personnel turnover at some sites numbered among the morale factors. All the studied agile pilot projects were evaluated as successful according to the company management. It was mentioned in interviews, for instance, that the development in the first agile pilot project was three times faster, with 50 times better quality and five times cheaper than previously in the company. The similar comments were received also from the managers of the pilot project two although the improvement was not as radical as in the project 1 compared to the company average situation.

4 Research Results

4.1 Results of the Within-Case Analysis

This section presents results of the within-case analysis for each of the case companies. All the barriers and strengths presented in Tables 5, 6, 7, and 8 were collected in workshops together with the organization representatives. Barriers and strengths were collected from individual team members (developers, managers, quality engineers, architects) in the series of workshops or interviews. Then, the team discussed if they all agree or disagree the argument. If the argument was valid for the whole team, the barrier or strength was further categorised together with the team (we counted how many same or similar individual comments were appeared) and explained in this paper using the in depth interview data.

Table 5 Categorised issues from the pilot post-mortem

Category	Case I	
	Strengths	Barriers
Agile architecture	0	10
Better understanding of the Customer demands	0	
Challenging technical environment	0	14
Communication and collaboration	7	4
Continuous estimation and planning	0	7
Continuous requirements management	2	17
Continuous project monitoring	0	0
Continuous testing	2	4
Domain knowledge	1	7
Learning	0	0
Improvement techniques	0	0
Self-organizing teams	0	0
UI process	4	1
Understanding of the roles and responsibilities	0	1
More lightweight documentation	0	1
Version control	1	7
Total	17	73

Table 6 Categorised issues from the agile assessments

Category	Case II	
	Strengths	Barriers
Agile architecture	0	0
Better understanding of the Customer demands	1	8
Challenging technical environment	0	16
Communication and collaboration	2	0
Continuous estimation and planning	1	16
Continuous requirements management	1	9
Continuous project monitoring	0	8
Continuous testing	0	0
Domain knowledge	0	0
Learning	0	0
Improvement techniques	0	0
Self-organizing teams	0	0
UI process	0	0
Understanding of the roles and responsibilities	0	0
More lightweight documentation	0	0
Version control	0	0
Total	5	57

Table 7 Categorised issues from the project post-mortem

Category	Case III	
	Strengths	Barriers
Agile architecture	0	0
Better understanding of the customer demands	3	
Challenging technical environment	0	0
Communication and collaboration	12	3
Continuous estimation and planning	7	6
Continuous requirements management	0	6
Continuous project monitoring	0	1
Continuous testing	6	9
Domain knowledge	0	2
Learning	2	0
Improvement techniques	2	0
Self-organising teams	9	0
UI process	0	0
Understanding of the roles and responsibilities	3	5
More lightweight documentation	5	7
Version control	0	0
Total	49	39

Table 8 Summary of the strengths and barriers in each case

Category	Case I		Case II		Case III	
	Strengths	Barriers	Strengths	Barriers	Strengths	Barriers
Agile architecture	0	10	0	0	0	0
Better understanding of the customer demands	0	0	1	8	3	0
Challenging technical environment	0	14	0	16	0	0
Communication and collaboration	7	4	2	0	12	3
Continuous estimation and planning	0	7	1	16	7	6
Continuous requirements management	2	17	1	9	0	6
Continuous project monitoring	0	0	0	8	0	1
Continuous testing	2	4	0	0	6	9
Domain knowledge	1	7	0	0	0	2
Learning	0	0	0	0	2	0
Improvement techniques	0	0	0	0	2	0
Self-organising teams	0	0	0	0	9	0
UI process	4	1	0	0	0	0
Understanding of the roles and responsibilities	0	1	0	0	3	5
More lightweight documentation	0	1	0	0	5	7
Version control	1	7	0	0	0	0
Total	17	73	5	57	49	39

4.1.1 Case I

In case company I, managers made the initial decision to launch an agile deployment within the organisation, resulting in the first pilot project in 2006. The project post-mortem and interviews conducted after the pilot revealed the strengths and barriers faced by the project team.

Select Agile Practices In the beginning, both management and pilot team demonstrated strong commitment to the deployment. The commitment was visible as active team participation while planning deployment activities. In general, the pilot team was satisfied with the management decision to test agile development practices in a pilot project. For example:

“I think it was great that there was the courage to start experimenting with this (agile).” (Developer, pilot project).

Plan Deployment Ten negative responses (i.e. barriers) were mentioned in the project post-mortem workshop, and interviews referred to the “Big Bang” approach selected for agile deployment. The project team found their scope insufficient owing to lack of clarity in the management vision regarding overall agile deployment. In addition, project developers lacked the necessary knowledge of agile development methods or the adopted application domain. Four negative remarks derived from the lack of training for the team prior to the project.

Resource allocation for the project was considered problematic. In practice, developers were allocated to multiple projects, which led to allocated resources being absent when critical decisions on the project activities were being made. As a consequence of these problems, some of the developers commented that the project was too large and complex for the launch of agile deployment. For example:

“I would first ... select some simple and smaller case, where the new process could be adopted in full.” (Developer, pilot project).

Execute Deployment The execution phase of the pilot was lengthy. However, neither iterative improvement workshop feedback nor any other SPI mechanisms were used to manage deployment and to improve practices in a systematic manner. As a result, no documented evidence exists for analysing whether or how the process was modified during the pilot. The lack of systematic improvements might be one reason why barriers kept emerging.

During the project post-mortem, however, it became evident that the planned Scrum- and XP-based pilot process had not been followed. It had actually remained unclear to the pilot team who was responsible for managing agile deployment, and how the defined process model was supposed to be implemented in practice. This uncertainty had led the team towards ad hoc deployment of practices they considered to be agile. For example:

“Wham bam, it was decided that this is agile and let’s do it like this. I think we should have let the developers implement it first and before that, naturally, set the definitions.” (Tester, pilot project)

Analyse, Improve and Package The decision to conduct a project post-mortem was actually made once the outputs of the project began to indicate problems regarding the schedule and

the quality of the project. Table 5 divides the 90 post-mortem findings into 14 categories, to illustrate the central strengths and barriers identified by the pilot team.

A total of 17 strengths were reported in the project post-mortem. The strengths concern the following categories (in descending order): 1) enhanced communication and collaboration with the project stakeholders, 2) applied UI procedures, 2–4) testing, and (2–17) management of the requirements and (1–7) version control. Enhanced communication was accomplished, e.g., via daily stand-up meetings, better team spirit and positive atmosphere affecting closer collaboration of developers, as well as due to the testers participation in the project since the beginning. In UI design, practices were put on place to enable earlier collaboration of designers and developers and early review and better use of UI scenarios, resulting as better compatibility of the user scenarios and UI layouts. Requirements management was seen to be improved due to the use of backlogs, and prioritization of the business needs due to the increased participation of the business stakeholders in the project. New version control tool was found beneficial for the project, yet version control of the UI layouts was still found inadequate. A total of 73 barriers were reported, addressing all categories. The top five problem areas identified are (in descending order): 1) requirements management, 2) technical environment, 3) architectural issues, 4) communication and collaboration and 5) a Testing. The project members found that too extensive, traditional project plans and in-front planning procedures were still applied causing confusion in the prioritization of requirements and continuous change of plans. In addition, poor effort estimation and lack of dependency management between requirements, made backlog management challenging. Multiple challenges were found in the project's technical environment, such as lack of CI tool support and unstable integration systems. Even though the overall team spirit and collaboration was found improved, challenges were found in communication due to the division of the project members in multiple projects, continuous change of plans and lack of clear responsibilities in the project. Challenges were also identified in testing, such as lack of unit-tests, testers' absence in review of user scenarios. The challenges in requirements engineering area affected challenges between the design and implementation and, therefore evidently also made the validation of the results and testing of the product difficult. Partially due to this, it was also reported that the error correction often resulted new errors in the code.

4.1.2 Case II

When agile deployment started, the company was following the plan-driven development approach, meaning that domain-engineering projects were based on the “freezing” of in-depth negotiations and requirements before any actual software development activities. The decision on agile deployment was made in 2005 on the understanding that part of the management group was not committed to the deployment of agile practices. The solution for agile deployment was a hybrid development model including both agile and plan-driven practices. The construction of the new model started by evaluating the software processes at company level, using CMMI goals and agile practices.

The agile deployment in Case II was divided into five main stages and spanned the period 2005–2008. The purpose of stage I was to identify suitable agile practices for the organisational level based on assessment strengths and improvement suggestions. The aim of stages II and III was for the teams to conduct the steps of agile deployment described below in selected pilot projects. During stage IV, the company used certain agile practices, such as sprints, sprint planning and post-game sessions, in five new pilot projects. The last workshop in stage V was conducted together with 16 software designers, architects and

managers, in order to assess the current state of agile deployment. As a result of the analysis, it was found that some agile practices were used in several projects, but that agile method use varied between the different types of projects (e.g. application and domain engineering). Analysis and selection of agile practices continued, taking place three times during the period 2005–2008. This was at the time management updated the company-level process models, based on the feedback collected from the pilot projects.

Select Agile Practices The first step in agile deployment in Case II was to organise training to supply the necessary information on agile methods (Mobile-D™, XP and Scrum) for the developers. Although the training sessions were planned successfully, some developers felt that project obligations prevented them from participating. A lack of awareness of agile methods caused a commitment problem among developers and management. For example:

“I had hoped that we would have an internal commitment to agile deployment. Now I’m the only one in our project with an awareness of agile methods, so the use of these methods remains a background [i.e. not actively used] activity” (Project manager, stages II and III).

The next step in agile deployment process was agile assessment, in which both project and requirements management process areas of CMMI were examined using a hybrid assessment framework, including both agile and plan-driven questions. The purpose was to increase awareness of agile methods and to facilitate overall communication and the company-level agile development process definition. The assessment resulted in the identification of 5 plan-driven and 12 agile improvement suggestions for the requirement management process area in CMMI. Furthermore, 17 plan driven and 35 agile improvement suggestions were related to CMMI project management (project planning, monitoring and control) process areas.

Plan Deployment Changes were made to the company-level agile development process model after agile assessment. The changes were made as a part of deployment planning.

Execute Deployment During the third stage of deployment, the selected agile practices were adopted in several new pilot projects. Use of agile practices was based on the freedom of the developer to select the agile practices and either a plan-driven or agile process model that they regarded as necessary for their work. Both commitment towards the change and the use of a new updated process model varied between individuals within the company. According to one project manager:

“We have used the process model, but it is often not possible to apply everything in practice, and it was difficult to use the process model as we don’t have tools to support the process.” (Project manager, stage III)

Analyse, Improve and Package The second and third evaluations of agile deployment status were carried out in 2006 and 2008 using an agile assessment, including the same hybrid plan-driven (CMMI-based) and agile (Scrum- and XP-based) question framework as in the previous analysis. It was found in these analyses that although agile practices were deployed in several new pilot projects, the agility level of the projects, measured by the number of agile practices used, varied greatly. The ratings from all the selected process areas—i.e. project planning, monitoring and control, and requirements management—showed that improvements were taking place in all process areas, from the average level P (partially achieved) to level L (largely achieved) or F (fully achieved).

Table 6 describes the number of strengths and barriers found in the second assessment divided into the different categories. Project managers tried to provide an enhanced understanding of agile development methods, sharing their own experiences of the practices adopted within the organisation. For example, developers and project managers had programme meetings once a week with project teams, to handle issues concerning project status and also to increase awareness of the use of agile practices.

The consequence of the overall deployment situation was perhaps increased awareness and the usage of agile methods, but also confusion among developers and project managers. The reasons for the confusion were the three main barriers to the use of agile practices: 1) difficult technical environment, which was claimed to be a limiting factor for continuous requirement changes, 2) continuous estimation and planning which was difficult to do in long term in the iterative planning environment. It was difficult because the developers need to work in several parallel projects and move between them continuously. Therefore, the plans that were made for the iteration often failed. (Table 6).

The first barrier revealed for technical environment was hardware-driven product development. For instance, a Senior Designer in the company stated that “*The co-development of software and hardware is...very difficult*” (Architect, Case II). The uses of hardware (more than software) made it difficult to integrate the system continuously and to synchronise development tasks for suitable pieces to develop in an iterative manner. Secondly, in many projects a technical solution had been implemented according to certain standards. The use of technical standards was cited as one of the main reasons for restricting the opportunity to change development direction later in the software development process.

The third key reason for agile practices not being used in all projects was related to customer demands. Here the customer often wanted to buy a product that fully covered all the requirements. The marketing people were therefore used to selling ready product packages, instead of ready pieces of the product as defined in the agile process.

Improvements were taking place, as projects using Scrum sprints, planning meetings and product backlogs seemed to be more focused, manageable and up-to-date than before in terms of change management for planning and requirements. The strengths of agile practices were clearly in the improved internal requirements management, more systematic ways to do communication and collaboration as well as the better understanding of the customer demands inside of the company. Agile practices made teams and project managers to come together regularly to discuss about the estimation and customer requirements, which helped teams to solve problems faster and in the end saved the time of the project managers.

4.1.3 Case III

Agile deployment started with agile practice selection and deployment planning for the selected pilot projects. As a result of the discussion the teams decided to pilot a combined set of Scrum, XP and Mobile-DTM (Abrahamsson et al. 2004)¹ practices. The piloting was performed in three pilot projects. Iterative improvement workshops (Salo and Abrahamsson 2007) played a key role as methods that directly supported the agile deployment process (i.e. an execution in three pilots and later at company level) but also worked as mechanisms for feedback collection from the project to organisational level. Parallel with the pilots, the

¹ Mobile-D is an agile method for mobile software development, developed by VTT Technical Research Centre of Finland.

managers (organisational level) started a company-level analysis of suitable agile practices. The feedback from the project-specific workshops and agile assessment was used as a part of the continuous analysis to generate a general company-level agile software development process description, including an illustration of “good” practices tailored for each project. In 2006, the new process model was adopted in all software development projects.

Select Agile Practices In 2003, managers in Case III made a decision to pilot agile methods as a way of responding to the needs of a dynamic market situation.

“In 2003, we had an event here in which some people from management participated, and at that point, we thought that we should try out Agile with a pilot project.”
(Manager, stage II)

Before the launch of the pilot project, the managers were interested in experimenting with different solutions based on the agile approach. In practice, management interest led to the situation where several agile practices from Scrum and XP were introduced to certain pilot projects at the same time as the Mobile-D™ process was deployed in another company project.

Plan Deployment As a first step towards agile deployment, company-wide training and mentoring of agile methods was organised for all project managers and software developers, to make them aware of the new approach (Scrum, XP and Mobile-D) for managing and implementing software: “over 2 weeks ...we were provided with training given by people from other companies” (Manager, stage II). Work in the entire company stopped during training, so that everyone would be involved. The information gained from the training prompted a number of project managers to try agile methods as a part of their software development process: “So, my idea was not to bring Scrum in, but rather to change the way we were working a little bit...” (Project Manager, stage II).

At the first stage, “the plan was to apply the new model in parallel with the plan-driven stage-gate process model” (Manager, stage II). After some time, however, the company discarded their plan-driven process model and defined a single model based on agile practices that were tailored to their own context. The scope of the process model was to extend the use of agile practices not only to development, but also to management and marketing levels.

Execute Deployment The deployment was conducted using the evolutionary approach as part of all projects in the company. In pilot 1, however, the entire project-level process was first changed based on Mobile-D™. In pilot 2, the project adopted practices from Scrum and XP (i.e. sprints, sprint planning meetings, daily meetings, iteration retrospectives, post-game sessions, continuous integration and test-driven development). In both pilot projects, iterative improvement workshops (i.e. iteration retrospectives) were used as a mechanism for evaluating the applicability of the agile practices selected, and to identify how the process could be improved. In practice, the developers had the freedom to select which agile practices to adopt during the next development iteration. In pilot 1, iterative improvement workshops were also used as a means of collecting continuous feedback from the project to organisational level.

Iterative improvement workshops and iterations in pilot 1 resulted in 26 issues and 44 strengths being defined and 27 improvement suggestions systematically implemented and validated. In pilot 2, iterative improvement workshops brought 54 improvement actions, 43 strengths and 113 improvement suggestions. However, the team did not apply mechanisms for systematically validating or reacting to the defined improvement ideas.

Analyse, Improve and Package Table 7 summarises the strengths and barriers that were evaluated in both project post-mortems and assessment workshops after 12 months of agile deployment.

The analysis evaluated the immediate application of daily communication and collaboration, continuous testing, continuous estimation and planning, more lightweight documentation, training and continuous improvement as well as the use of self-organising teams (who had the freedom to decide on the agile practices used) as strengths that motivated the company towards agile deployment. The communication, collaboration and the use of self organizing teams were also reported as factors that may increase the commitment of both management and developers towards the use of the agile approach. Based on these experiences, the Scrum master of another pilot project began to function as an agile change agent in the company, providing training for other managers on how to handle Scrum-based development projects. Other strengths, particularly from the deployment perspective, were 1) the iterative improvement workshops in which the team selected suitable agile practices for use in the next iteration and 2) an improvement in team spirit, which also facilitated the team's more efficient tackling of design problems.

More lightweight documentation was agreed to be strength especially among the development team because it saved the time of the developers to the coding activities. Continuous estimation and planning helped team members to create better understanding of the customer demands which was agreed to be a strength of agile deployment among the whole team. The estimation was, however, difficult at the beginning. It became easier when the developers got more use to plan hour level work activities and make estimations about their own work.

The negative issues were mostly related to the setting up of daily test and building environments and tools, where according to the developers, the management could provide more support during the early stages. Continuous unit testing was seen as a problem until iteration 6 in the project, where the iteration lasted one month. Building problems occurred continuously after iterations 3, 6 and 9. Communication and collaboration, project monitoring and documentation, requirements management were also evaluated as difficult regarding stakeholders that were as yet unaware of agile methods. Some stakeholders, for instance, expected agile team to produce architecture document that the team was ready to provide face to face, but not as a traditional written format. In the end the team had a workshop together with the stakeholders in which the document was produced. Stakeholders were also not yet aware of their roles and responsibilities in the new situation. They felt confused when the previous milestone procedure was changed to more continuous, and regular planning and monitoring activities.

Communication problems also increased at the end of the project as a result of the Scrum master change. Although a negative factor, communication was also evaluated as a positive issue in agile deployment. Most communication and collaboration barriers identified at this stage of agile deployment related to awareness, and interaction with other non-agile projects and other stakeholders.

4.2 Results of the Cross-Case Analysis

The cross-case analysis (summary in Table 8) showed that agile deployment strengths were mainly related to communication and collaboration inside the team and between the team and its stakeholders during deployment execution. Other identified strengths brought by agile deployment were unit testing improvements in Case II, requirements management and customer demand management improvements in Cases I and II, and planning and

estimation improvement activities in Cases II and III. Self-organising, improvement methods and learning aspects were only seen as strengths in Case III.

The agile deployment barriers differed between the cases analysed. The questions of communicating and collaborating with stakeholders in the new situation and of implementing test-driven development for the first time were seen as challenging, especially in Cases I and III. Estimation planning and technical environment (i.e. dealing with embedded product and standard functionality) parallel with agile deployment were reported as an issue, especially in Case II. In Case III, agile deployment barriers were recorded as also related to the roles and responsibilities of the new development approach, as well as of the more agile documentation approach.

From the agile deployment perspective we can also identify several reasons for these barriers emerging in the different cases. For example, in Case I, collaboration and communication did not work as planned because the team failed to apply the iterative planning approach. The reason the practice was not adopted by the team was that the team lacked the necessary knowledge and because management was not committed to organising or participating in planning meetings. In Case III, iterative planning was employed but not fully understood by the pilot project stakeholders.

The adoption of a new testing approach caused problems, especially in Case III. Although the management was committed to agile deployment in general, they failed to provide the necessary tools and training facilities for supporting the implementation of test-driven development. Conversely, in Case I, developers lacked knowledge of why and how to employ a test-driven development approach.

Deployment of an agile requirements management approach demands changes throughout the organisation. Merely to create a product backlog and establish sprint-planning meetings is usually insufficient. Continuous planning at project level also requires knowledge and understanding of how to carry out continuous planning at company strategy and road mapping level.

In Cases II and III it was found that estimation and planning is an agile practice that often demands additional training. Communicating to team members how to estimate work tasks for the next iteration should form part of iteration planning.

Technical environment, the use of standard functions, and embedded development environments forced Case II to tailor its agile practices. In these cases, it is important to define the use of agile development methods in specific situations.

The level of documentation in agile development is an issue much discussed in the agile development cases. The analysis revealed that the level of documentation is dependent on the needs of the stakeholders in the special context situations. For example, the large distributed embedded product in Case II demanded more documentation than the software product created by the co-located team in Case III. Management support is necessary in planning a suitable documentation level for agile development projects within the organisation.

Reported strengths in Case III were all related to the opportunity for developers to impact on their own work (self-organising team) and agile deployment planning (i.e. continuous improvement workshops).

As a summary of the analysis we can identify four recommendations that the management of a software company should take into account when planning the agile deployment. These are:

- 1) Make sure that the management is committed and gives continuous support for agile deployment
- 2) Make sure that both team, management and all stakeholders has a clear vision, understanding and *awareness* of agile methods

- 3) Give the needed freedom to the teams to tailor the agile methods to make the better fit to their specific needs
- 4) Make the continuous tailoring of the agile based process model also in the organizational level

5 Discussion

5.1 Implications and Recommendations to Practice

This section identifies implications and recommendations for agile deployment and discusses them with reference to the analysed cases and to the related literature.

5.1.1 Make Sure That the Management is Committed and Gives Continuous Support for Agile Deployment

The commitment of management is a solid and fundamental basis for changes (e.g. Niazi et al. 2006; Worley and Doolen 2006). Management support is vital in any major change in organisations; not only at the beginning of the change, but during the whole transformation (Worley and Doolen 2006). A lack of commitment is the factor most often mentioned in the failure of SPI programmes (Stelzer and Mellis 1998; Abrahamsson 2002).

In this research, commitment—or the lack of it—towards agile deployment has a substantial impact on its success. In Cases I and III, especially, the agile deployment process was launched and strongly supported by management. For example, even though a number of challenges were faced during the pilot (Case I), the team still identified management participation and the top-down deployment approach as being among the positive experiences gained from the pilot project. This, in turn, was found to affect the positive attitude and strong commitment of the team.

Strong management commitment, in Case I, is also visible through the fact that, even after 2 years of piloting considered as somewhat unsuccessful, they still made the decision to continue the deployment, but with a more rigorous approach to the second phase. In Case III, the close participation and commitment of management was visible through continuous improvement in the pilot process and in iterative feedback collection. While the development teams attempted to identify and solve process deficiencies throughout the pilot, management also participated where necessary in decision-making and action. Management were also positive about the new visibility and bridge created for them by the agile process during the development work. Iterative improvement workshops in Case III were considered to be a mechanism for improving team spirit and team commitment to the changes. This finding is in line with (Stelzer and Mellis 1998; Salo 2006), who argue that a successful change needs the continuous involvement of the development teams and communication between all the participants.

Börjesson et al. (2006) proposed that the deployment of improvement initiatives would demand an organisational-level pull and project-level push in order to succeed. However, the commitment of management and the development towards agile deployment do not always go hand in hand. For example, In Case II, part of the management was highly committed, while another part was resistant to the change. This was still found to be the case 4 years after the launch of agile deployment. At the same time, all developers seemed committed to using agile methods based on the issues emerging from the second agile assessment. The lack of management commitment led to a situation in which the use of

agile methods varied between projects and developers were confused as to whether they should use agile methods or not.

5.1.2 Make Sure That the Management Has a Clear Vision, Understanding and Awareness of Agile Methods

Awareness and information sharing have been considered to be an important part of SPI implementation (Niazi et al. 2006). Training is one way to increase developer's awareness and understanding of agile development methods. Training, and highly skilled staff, have recently been reported as critical factors for successful SPI (Niazi et al. 2006). In fact, SPI initiatives cannot be implemented without a complete understanding of the necessary details (Niazi et al. 2006).

According to our study, training is also important when deploying the agile approach. The extent of the promotion of agile methods was, in fact, revealed as one of the main factors affecting the success of deployment within the case companies. Again, we found that the three case companies differed greatly from each other. For example, there was practically no training arranged in Case I, participation in training was voluntary in Case II, and the entire personnel was trained beforehand in Case III. The efficient training and coaching phase can be claimed to have affected the successful adaptation of agile methods in Case III, where agile practices had spread throughout the company significantly faster than in the other cases.

Thus, in the first two cases, the awareness and understanding of agility was to be built during the deployment process. Problems due to the lack of awareness were encountered, however, for example in the practical application of agile methods during the pilot (Case I) and resistance towards the adoption (Case II). The results of the training are in the line with the findings of (Misra et al. 2009), who argue that training has a significant correlation with success.

5.1.3 Give the Needed Freedom to the Teams to Tailor the Agile Methods to Make the Better Fit to Their Specific Needs

We also found that the successful deployment of improvement initiatives would demand a certain level of freedom among developers to decide on adoption of agile practices. In Case I, where only one pilot was under way at the time of the research, freedom of choice was not an issue and only concerned the stakeholders for the pilot project. In Case II, the organisation endowed development teams with the freedom to participate in training and to choose between the existing plan-driven development process and the new process model with agile modifications. In Case III, the first pilot team was mandated to follow the Mobile-D method by the book. On the other hand, in the later pilots, developers and managers had the freedom, though only at first, to select and tailor the agile practices. However, application soon became mandatory for all once the new agile process model was adopted throughout the company. In comparing the results for Case II and III, we argue that a high level of freedom does not always ensure the effective and fast deployment of agile methods. On the contrary, a mandatory involvement in agile training, for example, can be defined as a driver towards, rather than a factor for, agile deployment, which may cause additional resistance among the teams.

Since the participation of employees is important, it is also one of the fundamental ideas presented in many improvement models (Dybå et al. 2004), such as QIP. Dybå et al. (2004) state that there should always be the opportunity for employees to improve their work and feel the sense of contribution at an organisational level. In addition, the agile principles highlight the continuous adaptation process taking place in individual project teams (Beck et al. 2001), while research also claims multiple positive implications in integrating the

iterative improvement and feedback in agile teams and organisational learning (Salo 2006). Misra et al. (2009) also state that there might be a strong correlation between continuous learning and agile development success. In our study, the most efficient application of multiple SPI mechanisms was achieved in Case III, where the assessments, pilot post-mortems and continuous systematic iterative improvement workshops were used as a feedback mechanism from project to organisational levels. Efficient feedback mechanisms facilitated the rapid construction of the organisational-level agile process model as well as the rapid learning of the development team. On the other hand, in Case I it was only at the end that a pilot post-mortem was conducted, this failing to unearth any evidence of process improvement during the pilot.

5.1.4 Make the Continuous Tailoring of the Agile Based Process Model Also in the Organizational Level

The three case companies took different approaches in defining the base process for agile deployment. In Case I, there was no clearly defined or documented company-wide process model to support agile deployment. Owing to the lack of an agile development model, the pilot post-mortem revealed severe shortcomings in the process model being piloted. In Case II, the agile process was created by tailoring the company's plan-driven process model through the adding of agile practices. In the first pilot of Case III, an existing agile process model, i.e., Mobile-DTM (Abrahamsson et al. 2004) was taken up and adapted. It seems that although agile methods bring some mechanisms for improving software development, this may not be enough for efficient software development at all levels. Therefore, it is important that the company develops an agile software development process model in parallel with agile deployment, from the feedback continuously harvested from the project-level iterative improvement workshops. This is in line with Kettunen (2009), who suggests that agile methods should be transferred from project to organisational levels and even to enterprise-wide value chains. However, Williams and Cockburn (2003) claim that software development cannot be considered to be a defined process, as changes occur during the development time. They continue that it is highly unlikely for any set of predefined steps to lead to a desirable, predictable outcome because requirements change, technology changes, people are added and taken off the team, and so on. Baskerville et al. (2006) claim that tailored processes and methods are useful for the development teams. By keeping the development process fluid, high-speed development absorbs environmental changes easily. Additionally, they say that a process with too much fluidity could degenerate into no process at all.

5.2 Implications to Agile Deployment

In all the three cases, the use of agile practices were still in the deployment phase since 1) the agile practices were considered new for the adopters, 2) agile practices were not yet seen as a part of the organization routines or process model and 3) the agile practices were not yet used in the large amount of projects in the case organization. However, the situation was different between the studied organizations and changed over the time when the research was conducted. For example workshops in Case II revealed that the agile practices were used only in one pilot project first but in after 3 years, the usage was broaden also into five other projects in the same organization. The same happened in the case organization III in which the agile practices were in use in the whole company after 5 years of deployment.

In both cases II and III, the company representatives made a change to the general process guidelines relatively fast in order to support the agile deployment activities. It,

however took 2 to 3 years before the developers actually adopted the new process model as a part of their daily practices. In this adoption, the post iteration workshops worked as a useful mechanism to help team to see what needs to be done in order to support the continuous improvement. Case company I failed agile piloting first but was still interested to continue the deployment after the more in depth analysis and understanding of the reasons behind the failure. After the post project workshops, and use of specialist help they recognised that the practices that they used to call as agile were only ad hoc practices that were deployed in the pilot project without real knowledge about the usage of the agile methods.

6 Conclusions and Further Work

This paper focuses on identifying the central success factors and recommendations for agile deployment in software companies. The study examined the implementation of agile deployment in three case companies over a 2.5 year period. As a result, 71 strengths and 169 barriers of agile deployment were identified. The analysis revealed the importance of management providing the necessary goals and support for agile development. It also indicated the significance of defining a tailored process model and giving developers the freedom to improve their own agile development process continuously during agile deployment.

A limitation of this research is that the data collected are mainly based on the subjective opinions of interviewees from the companies that were studied. It may also be difficult to compare issues and strengths collected from companies owing to the varying techniques used for data collection. The techniques selected (other than semi-structured interviews) were based on the current needs of the firms involved. Agile assessments were employed, however, in all the case companies, although at slightly different phases of agile deployment. One strength of this study is the richness of the data and the evident long-term view of agile deployment within three different domain areas of software development. It may prove difficult, however, to repeat the results of the research owing to varying context and highly impacting environmental factors. In the future, additional research will be necessary for clarifying the means by which organisations can select a suitable deployment strategy that leads to successful agile deployment. Another goal of future study might be to address the deployment aspect of agile methods, especially in the context of large and multi-site development contexts. The identified barriers and strengths might be examined from the perspective of their applicability for organisational change and organisational behaviour. For example, the literature or theories of organisational behaviour might be investigated to see if the barriers and strengths identified in this study are a particularly common pattern for any kind of organisational change.

Acknowledgments This work was carried out in AGILE-ITEA2 and FLEXI-ITEA2 projects funded by TEKES (the Finnish Funding Agency for Technology and Innovation).

References

- Abrahamsson P (2002) The role of commitment in software process improvement. Dissertation, University of Oulu

- Abrahamsson P, Hanhineva A et al (2004) Mobile-D: an agile approach for mobile application development. Conference on Object Oriented Programming Systems Languages and Applications, Vancouver, Canada
- Basili VR (1989) Software development: a paradigm for the future. COMPSAC '89, Orlando, Florida
- Baskerville R, Ramesh B et al (2006) High-speed software development practices: what works, what doesn't. IT Prof 8(4):29–36
- Beck K, Andres C (2005) Extreme programming explained. Addison-Wesley Professional
- Beck K, Beedle M et al (2001) Manifesto for agile software development. <http://AgileManifesto.org>. Accessed 15 October 2010
- Benbasat I, Goldstein DK et al (1987) The case research strategy in studies of information systems. MIS Q 11(3):369–386
- Boehm B (2002) Get ready for agile methods, with care. IEEE Comput 35(1):64–69
- Boehm B, Turner D (2003) Using risk to balance agile and plan-driven methods. IEEE Comput 36(6):57–66
- Börjesson A, Martinsson J et al (2006) Agile improvement practices in software organizations. Eur J Inf Syst 15:169–182
- Chan FKY, Thong JYL (2008) Acceptance of agile methodologies: a critical review and conceptual framework. Decis Support Syst 46:803–814
- Cohen D, Lindvall M et al (2004) An introduction to agile methods. Elsevier Academic Press
- Drobka J, Noftz D et al (2004) Piloting XP on four mission critical projects. IEEE Softw 21(6):70–75
- Dybå T (2005) An empirical investigation of the key factors for success in software process improvement. IEEE Trans Softw Eng 31(5):410–424
- Dybå T, Dingsøyr T (2008) Empirical studies of agile software development: a systematic review. Inf Softw Technol 50(9–10):833–859
- Dybå T, Moe NB et al (2004) An empirical investigation of factors affecting software developer acceptance and utilization of electronic process guide. 10th International Symposium on Software Metrics (Metrics'04)
- Eisenhardt K (1989) Building theories from case study research. Acad Manag Rev 14(4):532–550
- Fishman RG (1992) Information technology diffusion: a review of empirical research. MIT Sloan School of Management. 50 Memorial Drive, 53–314
- Fitzgerald B, Hartnett G et al (2006) Customising agile methods to software practices at Intel Shannon. Eur J Inf Syst 15(2):200–213
- Garvin DA (2000) Learning in action. Harvard Business School Press, Boston
- Hansson C, Dittrich Y et al (2006) How agile are industrial software development practices? J Syst Softw 79:1295–1311
- Kähkönen T (2005) Life cycle model for software process improvement project deploying an agile method. International Conference on Agility, Helsinki
- Kähkönen T, Abrahamsson P (2004) Achieving CMMI level 2 with enhanced extreme programming approach. Profes, Japan
- Kettunen P (2009) Adopting key lessons from agile manufacturing to agile software and product development—a comparative study. Technovation 29:408–422
- Kettunen P, Laanti M (2005) How to steer an embedded software project: tactics for selecting the software process model. Inf Softw Technol 47(9):587–608
- Lindvall M, Muthig D et al (2004) Agile software development in large organizations. Comput Pract 37(12):38–46
- McCaffery F, Richardson I et al (2006) Adept—a software process appraisal method for small to medium-sized Irish software development organisations. EuroSPI06, Finland, Joensuu
- McCaffery F, Pikkarainen M et al (2008) AHAA—agile, hybrid assessment method for automotive, safety critical SMEs. ICSE 2008, Leipzig, Germany
- McCaffery F, Pikkarainen M et al (2009) AHAA: integrated assessment tool for plan-driven and agile processes. In: Maxwell GA, Drummond SK (ed) Automotive industry: technical challenges, design issues and global economic crisis
- McFeeley B (1996) IDEAL(SM): a user's guide for software process improvement. CMU/SEI-96-HB-001. Software Engineering Institute (SEI)
- Miles M, Huberman A (1999) Qualitative data analysis. Sage, London
- Misra SC, Kumar V et al (2009) Identifying some important success factors in adopting agile software development practices. J Syst Softw 82(11):1869–1890
- Niazi M, Wilson D et al (2006) Critical success factors for software process improvement implementation: an empirical study. Software Process Improv Pract 11:193–211
- Patton MQ (2002) Qualitative research & evaluation methods. Sage Publications. 452–455
- Paulk MC (2001) Extreme programming from a CMM perspective. Software 18(6):19–26

- Petersen K, Wohlin C (2010) The effect of moving from a plan-driven to an incremental software development approach with agile practices. An industrial case study. *Empir Softw Eng* 15:654–693
- Pikkarainen M, Mäntyniemi A (2006) An approach for using CMMI in agile software development assessments: experiences of three case studies. *SPICE 2006*, Luxembourg
- Pikkarainen M, Salo O et al (2005) Deploying agile practices in organizations: a case study. *European Software Process Improvement and Innovation, EuroSPI*, Budapest, Hungary
- Rasmusson J (2003) Introducing XP into Greenfield projects: lessons learned. *IEEE Softw* 20(3):21–28
- Robson C (2002) *Real world research*, 2nd ed. Blackwell Publishing
- Royce W (1970) Managing the development of large software systems. *IEEE WESCON*
- Salo O (2006) Enabling software process improvement in agile software development teams and organizations. Dissertation, VTT Technical Research Centre of Finland, University of Oulu
- Salo O, Abrahamsson P (2007) An iterative improvement approach for agile development: implications from multiple case study. *Software Process Improv Pract* 12(1):81–100
- Schwaber K, Beedle M (2002) *Agile software development with scrum*. Prentice-Hall, Upper Saddle River
- Stelzer D, Mellis W (1998) Success factors of organizational change in software process improvement. *Software Process Improv Pract* 4(4):227–250
- Svensson H, Höst M (2005) Introducing an agile process in a software maintenance and evolution organization. 9th European Conference of Maintenance and Reengineering, Manchester, UK
- Sweeney A, Bustard DW (1997) Software process improvement: making it happen in practice. *Softw Qual J* 6(4):265–274
- Williams L, Cockburn A (2003) Agile software development: it's about feedback and change. *Computer* 36(6):39–43
- Williams L, Layman L, Krebs W (2004) Extreme programming evaluation framework for object-oriented languages. *Computer Science TR-2004-18*
- Worley JM, Doolen TL (2006) The role of communication and management support in a lean manufacturing implementation. *Manag Decis* 44(2):228–245
- Yin RK (2003) *Case study research: design and methods*. Thousand Oaks



Dr. Minna Pikkarainen is a Principal Research Scientist in VTT Technical Research Centre of Finland. She has worked in several industrial-driven research projects and project preparations doing close industrial collaboration with large amount of organizations in Europe since 1997. Pikkarainen has been a member of Lero (The Irish Software Engineering Research Centre) during the years 2006–2009. Currently, she is a member of Siris, The Collective Center for the Belgian technological industry. Recently, her work and numerous publications have mainly focused on research in the areas of agile development and software innovation.



Dr. Outi Salo has been working for Nokia Corporation since 2008 in various managerial roles of operational development. Previously, she worked at VTT Technical Research Centre of Finland as a Senior Research Scientist (2001–2008) and as a System Analyst at Elbit Oy (1998–2000). Outi's main interest areas of research and practice involve agile software product development, software process improvement, business process re-engineering and metrics. Currently, she is also finalizing her studies to become a qualified vocational teacher in software engineering. Outi received her Ph.D. in Information Processing Science from the University of Oulu, Finland in 2007.



Dr. Raija Kuusela has almost 30 years' experience of the ICT sector. She has been working as a senior research scientist at VTT since 2009. Prior to that, she worked for Nokia for 14 years in several roles including software development roles, Engineering Manager, Quality Manager, Scrum Master, and Product Owner. Before her career at Nokia, Kuusela worked with other software intensive companies for 13 years in software development roles. She received her PhD in Industrial Engineering and Management from the University of Oulu, Finland in 2007.



Dr. Pekka Abrahamsson is a full professor of computer science at Free University of Bozen-Bolzano in Italy. He has held academic positions in University of Oulu, University of Tampere, SITNEF, VTT Technical Research Centre of Finland and most recently in University of Helsinki. He received his PhD from University of Oulu in 2002 on software process improvement. His research interests are centered on empirical software engineering, agile & lean software development and cloud computing. He has lead large European research projects on these topics. He has published actively in scientific conferences and journals. He is the recipient of the Nokia Foundation Award in 2007. He is a member of the IEEE, ACM, and ISERN. His practical experience involves five years in the software industry as a software developer and a quality manager.